AI Accelerator Design – Assignment 3: BCHW Tensor Flattening and Reconstruction

Name: Abhishek Kumar Maurya Roll No: CS25M201

1. Address mapping

The tensor I has shape B×C×H×W and is stored row-major (W varies fastest). Each sample b becomes one row of the 2-D tensor F of shape B×(CHW). Inside a row, element (c,h,w) is placed at linear address **k = c·HW + h·W + w**, with 0 ≤ k < CHW. The inverse uses integer division and modulo: **c = k div HW, h = (k mod HW) div W, w = k mod W**. The batch index b is the row index in both representations. No reshape, view, flatten or ravel is used in the algorithm; arrays are only allocated with empty() and accessed element by element.

2. Pseudo code

```
FLATTEN(I, B, C, H, W):
  F <- empty(B, C*H*W)
  for b in 0..B-1
    for c in 0..C-1
      for h in 0..H-1
        for w in 0..W-1
          k <- c*H*W + h*W + w
          F[b, k] <- I[b, c, h, w]
  return F
```

```
RECONSTRUCT(F, B, C, H, W):
  R <- empty(B, C, H, W)
  for b in 0..B-1
    for k in 0..C*H*W-1
      c <- k div (H*W)
      h <- (k mod (H*W)) div W
      w <- k mod W
      R[b, c, h, w] <- F[b, k]
  return R
```

3. Manual indexing example (B=1, C=2, H=2, W=3, so HW=6)

Values were set to 10(c+1)+3h+w so each element is easy to identify.

(b,c,h,w)	k = 6c + 3h + w	value	reverse: (k//6, (k%6)//3, k%3)
(0,0,0,0)	6*0+3*0+0 = 0	10	(0,0,0)
(0,0,0,2)	6*0+3*0+2 = 2	12	(0,0,2)
(0,0,1,0)	6*0+3*1+0 = 3	13	(0,1,0)
(0,0,1,2)	6*0+3*1+2 = 5	15	(0,1,2)
(0,1,0,0)	6*1+3*0+0 = 6	20	(1,0,0)
(0,1,0,1)	6*1+3*0+1 = 7	21	(1,0,1)
(0,1,1,2)	6*1+3*1+2 = 11	25	(1,1,2)

Resulting flattened row: [10, 11, 12, 13, 14, 15, 20, 21, 22, 23, 24, 25]. Channel 0 occupies k = 0–5 and channel 1 occupies k = 6–11; within each channel the rows of length W=3 are laid end to end. Reconstruction returned the original tensor exactly.

4. Experiments and results

B = 4 for every case. MNIST and CIFAR-10 test images use their native sizes; synthetic feature maps use H = W = 16 with values drawn from N(0,1) (float32), with C swept from 8 to 500. Errors are computed as E = I - R (R = reconstructed tensor) over all elements.

Dataset/Input	B	C	H	W	CHW	Emax	MAE
MNIST / test samples	4	1	28	28	784	0	0
CIFAR-10 / test samples	4	3	32	32	3072	0	0
Synthetic fmap1	4	8	16	16	2048	0	0
Synthetic fmap2	4	16	16	16	4096	0	0

Synthetic fmap3	4	32	16	16	8192	0	0
Synthetic fmap4	4	64	16	16	16384	0	0
Synthetic fmap5	4	128	16	16	32768	0	0
Synthetic fmap6	4	256	16	16	65536	0	0
Synthetic fmap7	4	500	16	16	128000	0	0

As an independent check (clearly separated in the code and not part of the algorithm), the explicit flatten output was compared with NumPy's reshape(B, -1) and matched exactly.

5. Discussion

Both Emax and MAE are exactly zero for every configuration, from C = 1 to C = 500. This is expected: the forward map k = c·HW + h·W + w is a bijection between {0..C-1}×{0..H-1}×{0..W-1} and {0..CHW-1}, and the div/mod reverse map is its exact inverse, so every element is copied once to a unique location and copied back with no arithmetic on the values themselves (no rounding). The channel count only changes the length of each row (CHW), so correctness does not depend on C; only run time grows, roughly linearly with B·C·H·W because the loops touch every element once in each direction. In an accelerator, this same address generation is what a DMA or address-generation unit performs when streaming a feature map into a linear buffer or feeding a systolic array: consecutive k addresses walk along W, then H, then C, so data from one channel stays contiguous.